OS Lab Assignment 7 — Threads, Semaphores, Deadlock, Priority Scheduling in xv6

Building and Running

```
make qemu-nox            # 2 CPUs (used for Q1–Q3 and the 2-CPU Q4 run)
make qemu-nox CPUS=1     # 1 CPU (used for the main Q4 run)
```

Inside xv6, run `threadtest`, `synctest`, `deadlock` and `priotest`. The terminal output of every run is in `output_logs/`.

Folder	Contents
`kernel/`	`proc.c`, `proc.h`, `sysproc.c`, `syscall.c`, `syscall.h`, `defs.h`, `param.h`, `trap.c`
`user/`	`threadtest.c`, `synctest.c`, `deadlock.c`, `priotest.c`, `user.h`, `usys.S`
`Makefile`	adds the four programs to `UPROGS`, the `-smp N,sockets=N` fix, and drops `-Werror`
`output_logs/`	per-question terminal logs and the full 2-CPU session

New system calls: `clone` (26), `join` (27), `set_priority` (28) and `get_priority` (29). `sem_init` (23) and `sem_wait` (24) were reimplemented, and `sem_signal` (25) replaces A5's `sem_post`.

Q1 — Kernel-supported threads (`clone` / `join`)

Design and data structures. `struct proc` gained `int isthread`. `clone(fcn, arg, stack)` works much like `fork()`. It calls `allocproc()` to get a fresh PCB and kernel stack, but sets `np->pgdir = curproc->pgdir` so the address space is **shared**, not copied. On the user stack page passed in by the caller it writes a fake return address `0xffffffff` and then `arg`. It then sets `tf->esp` to that frame and `tf->eip = fcn`. Open files and the cwd are duplicated, the same way `fork()` does it. `sys_clone` uses `argptr(2, &stack, PGSIZE)`, so the kernel rejects a stack that doesn't lie entirely inside the process's memory.

`join()` mirrors `wait()`, but it only reaps children with `isthread == 1`. It frees the kernel stack and **never calls `freevm`**, because the page directory still belongs to the parent.

Changes to `exit` / `wait` / `fork`:

- `wait()` skips thread children, so a process can't accidentally free a shared pgdir. It calls `freevm()` only when `!p->isthread`. The exception is `init`, which may reap orphaned threads, and it does so without `freevm`.
- `exit()` marks the exiting process's still-running threads as `killed` when it hands them to `init`.
- `fork()` called from a thread copies the shared pgdir with `copyuvm`, so it needs no change.

Result (before synchronization), 3 threads × 100,000 increments, 2 CPUs:

Run	Final counter	Lost updates
1	271,794	28,206
2	211,379	88,621
3	223,534	76,466

The parent sees every thread's increments in its own `counter`, which shows the address space really is shared. The losses come from the non-atomic `load / add / store` of a `volatile int` running on two CPUs at once.

Q2 — Counting semaphores

Design and data structures. In `proc.c`:

```
struct semaphore { int value; int wakeups; struct spinlock lk; };
struct { struct spinlock lock; struct semaphore sem[NSEM]; } semtable;   /* NSEM = 10 */
```

`pinit()` initializes all the locks. `sem_init` takes the table lock and then the semaphore's lock. `sem_wait` decrements `value`, and if the result is `< 0` it sleeps on the semaphore with `sleep(s, &s->lk)` until `wakeups > 0`, then consumes one wakeup. `sem_signal` increments `value`, and if the result is `<= 0` it posts one wakeup and calls `wakeup(s)`. A negative `value` therefore counts the blocked processes, as the spec defines. The `wakeups` counter means only one sleeper gets through per signal, even though xv6's `wakeup()` wakes everyone on the channel.

Before/after comparison of the race condition:

Program	Synchronization	Final counter
`threadtest`	none	211,379 – 271,794 (wrong, varies per run)
`synctest` part 1	`sem_wait(0)` / `sem_signal(0)` around `counter++`	**300,000** (exact)

Producer–Consumer (`synctest` part 2): one producer thread and one consumer thread share a 5-slot circular buffer, with `empty = 5`, `full = 0` and `mutex = 1`. Twenty items were produced and consumed in FIFO order. The printed buffer count always stayed within `0..5`: the producer blocks at 5 and the consumer blocks at 0. Final result: `count=0, overflow=no, underflow=no`.

Q3 — Deadlock detection and the Banker's algorithm

Design. `deadlock.c` keeps static arrays `avail[m]`, `alloc[n][m]`, `request[n][m]`, `max[n][m]` and `need[n][m]`.

- `detect_deadlock(int *list)` starts with `Work = Available`, and with `Finish[i] = true` for any process that holds nothing. It then repeatedly looks for an unfinished process whose `Request ≤ Work`, adds that process's allocation back into `Work`, and prints each step. It returns how many processes are deadlocked and fills `list` with their indices.
- `is_safe_state()` is the Banker's safety algorithm run on `Need = Max − Allocation`. It prints either the safe sequence or `UNSAFE`.
- `request_resources(p, req)` checks `req ≤ Need` and `req ≤ Available`, allocates tentatively, calls `is_safe_state()`, and rolls the allocation back if the state would be unsafe.
- For the real deadlock, the parent creates two pipes and forks A and B. A keeps Pipe1's write end and blocks reading Pipe2; B keeps Pipe2's write end and blocks reading Pipe1. Each child reports over a status pipe before it blocks. The parent waits 100 ticks, builds the allocation and request matrices for the two pipes, runs detection, and then kills both children.

Results:

Test	Matrix	Expected	Program output
1	5×3, Request from the textbook example	no deadlock	`no deadlock` (order P0, P2, P3, P4, P1)
2	same, but P2 requests one more C	deadlock	`DEADLOCK among P1 P2 P3 P4`
3	Banker's, Available = (3,3,2)	safe	`SAFE <P1 P3 P4 P0 P2>`; P1's request (1,0,2) **granted**
4	after test 3, P0 requests (0,2,0)	becomes unsafe	`UNSAFE`, request **denied** and rolled back
5	real processes A and B on 2 pipes	circular wait	`DEADLOCK (circular wait) among A B`

Available, Allocation and Request/Max/Need are printed at every step (see `q3_deadlock.txt`).

Q4 — Priority-based preemptive scheduler with aging

Design. `struct proc` gained `priority` (1 is highest, 20 is lowest; `allocproc` sets the default of 10) and `waitticks`.

- `scheduler()` scans the whole table, starting just after the last slot it picked (`last`, guarded by `ptable.lock`), and runs the RUNNABLE process with the smallest priority number. Because it only switches to a strictly smaller number, ties go to whichever process comes first after `last`, which gives round-robin among equals.
- Preemption comes from xv6's existing timer-driven `yield()`: on every tick the scheduler runs again and picks the best process.
- Aging: `trap.c` calls `ageprocs()` on every timer tick. That function increments `waitticks` for every RUNNABLE process. Once a process passes `AGE_TICKS = 30`, its priority number drops by 1 (never below 1) and its counter resets. Being scheduled also resets the counter.
- `set_priority(pid, prio)` checks that `prio` is between 1 and 20. `get_priority(pid)` returns −1 for an unknown pid.

`priotest` forks four identical CPU-bound children with priorities 2, 6, 12 and 20. The parent runs at priority 1 so it isn't delayed while forking. A monitor process, also at priority 1, polls `get_priority()` for the priority-20 child and prints each aging step.

Completion time vs. priority (`CPUS=1`, `q4_priotest_1cpu.txt`):

Rank	Start priority	Completion (ticks)	Final priority
1	2	147	2
2	6	293	2
3	12	439	3
4	20	581	6

The completion order matches the priority order exactly. The priority-20 process aged one level every 30–32 waiting ticks (20 → 19 at tick 34, …, 7 → 6 at tick 436), then got the CPU and finished. It did not starve.

On 2 CPUs (`q4_priotest_2cpu.txt`) the order is still 2, 6, 12, 20 (144 / 156 / 287 / 305 ticks). The two highest-priority jobs run in parallel, and the last two get CPUs once the first two finish.

Problems faced and how they were resolved

1. **No race visible at first, and only `cpu0` booting.** QEMU 10 turns `-smp 2` into one socket with two cores, and the MP table that xv6 parses then lists a single CPU. I found this by printing `ncpu` at boot, which showed 1. The fix was `-smp $(CPUS),sockets=$(CPUS)` in the Makefile, after which `cpu1: starting 1` appears.
2. **Threads finished before the next one started**, so no increments overlapped. I added a `volatile int go` start barrier that the parent sets after creating all the threads.
3. **Build failure with GCC 15.** `-Werror` turned the `-Warray-bounds` warnings in stock `mp.c` into errors. I removed `-Werror`, matching how the earlier assignments were built.
4. **Lost wakeups / thundering herd.** xv6's `wakeup()` wakes every sleeper on a channel, so the semaphore keeps a `wakeups` count and lets exactly one sleeper through per signal.
5. **`struct spinlock` inside `proc.h`** broke files that include `proc.h` before `spinlock.h`. I moved `struct semaphore` into `proc.c`, its only user.
6. **`priotest` hung after the first two jobs.** Each child used to loop with `sleep(1)` until its priority equalled the value assigned to it. Aging changed the priority while the child waited, so the loop never ended. I removed the handshake: the priority-1 parent sets each child's priority right after `fork()`.
7. **Interleaved output** from the two deadlock children running on separate CPUs. The parent now forks B only after A has reported.

Known limitation: if a process exits while its threads are still running, the address space is freed once the parent reaps it, even though the threads are already marked killed. All the test programs `join()` their threads first.